

■ Contability Demo

Documentación Técnica Completa

Sistema de contabilidad personal en PHP puro con arquitectura MVC, Query Builder propio y conexión a SQL Server mediante PDO.

Stack	PHP 8.1+ SQL Server Bootstrap 5 jQuery Chart.js
Patrón	MVC + Front Controller + Singleton + Fluent Interface
Versión	Demo (GitHub)
Licencia	Demo educativo — libre para uso y aprendizaje

1. Arquitectura del Proyecto

El proyecto sigue una arquitectura MVC (Model-View-Controller) implementada sin frameworks externos. Toda petición HTTP ingresa por **index.php** y es enrutada por **App::run()** antes de llegar a cualquier controlador.

Estructura de Directorios

Ruta	Descripción
index.php	Punto de entrada único. Carga config y arranca el router.
Autoload.php	Autoloader PSR-4 personalizado. Mapea namespaces a rutas.
.htaccess	Redirige todo a index.php para URLs amigables (Apache).
app/App.php	Router / Front Controller. Decide qué controlador invocar.
app/login.php	Vista HTML del formulario de autenticación.
app/config/app.php	Config activa: sesión, constantes, DB_SERVERS. En .gitignore.
app/config/app.example.php	Plantilla de configuración para el repositorio.
app/database/Connection.php	Singleton PDO con soporte de múltiples servidores.
app/database/Database.php	Query Builder encadenable (Fluent Interface).
app/helpers/Auth.php	Control de acceso por sesión y por roles.
app/controllers/	HomeController, UserController, AjaxController.
app/models/	9 modelos. Cada uno extiende Database.
app/views/	Vistas PHP + assets (Bootstrap, FontAwesome, Chart.js).

2. Flujo de Peticiones

Cada petición HTTP sigue este recorrido antes de generar respuesta:

Paso	Componente	Acción
1	index.php	Carga app.php (config, sesión, DB_SERVERS) y Autoload.php
2	App::run()	Detecta entorno, verifica sesión, parsea URL
3A	AjaxController	Si controller=Ajax → responde JSON (requiere sesión)
3B	UserController::loginAction()	Si no hay sesión → muestra formulario de login
3C	{Controller}::{action}Action()	Si hay sesión → carga head.php, ejecuta, carga footer.php

3. Capa de Base de Datos

3.1 Connection.php — Singleton PDO

Implementa el patrón **Singleton**: la conexión PDO se crea una sola vez y se reutiliza en toda la petición. Soporta múltiples servidores definidos en **DB_SERVERS** (app.php), intentándolos en orden si uno falla (fallback automático).

Driver requerido: php_pdo_sqlsrv (habilitarlo en php.ini)

3.2 Database.php — Query Builder (Fluent Interface)

Clase base que extienden todos los modelos. Permite construir consultas SQL encadenando métodos. Todos los valores se pasan como parámetros posicionales (?) usando Prepared Statements — nunca hay interpolación directa de variables en el SQL.

Método	SQL generado	Uso típico
get(\$table, \$cols)	SELECT cols FROM table	Consultar registros
set(\$table, \$cols)	INSERT INTO table (cols) VALUES (?...)	Insertar registro
put(\$table, \$cols)	UPDATE table SET col=?...	Actualizar registro
delete(\$table)	DELETE FROM table	Eliminar físicamente
where(\$cols,\$ops,\$logic)	WHERE col1 OP ? AND col2 OP ?	Filtrar resultados
join(\$tbl,\$on,\$col,\$type)	INNER/LEFT JOIN tabla ON ...	Unir tablas
whereBetween(\$col)	WHERE/AND col BETWEEN ? AND ?	Filtrar por rango
execute(\$params)	Ejecuta y retorna resultados	SELECT→array, resto→bool

Importante — orden de parámetros en execute(): Los valores se pasan en el mismo orden en que aparecen los '?' en el SQL ensamblado: primero los del WHERE, luego los del BETWEEN. Respetar este orden es crítico para que los filtros funcionen.

4. Modelos

Cada modelo extiende **Database** y encapsula las consultas de su tabla. Los controladores solo llaman métodos de modelo; nunca escriben SQL directamente.

Modelo	Tabla	Métodos principales
UserModel	Usuarios	getUser(nombre, activo)
TransacionModel	Transacciones	getmontomes, getmontomesegresos, getbalancetotal, getVerTransacciones, setTransaction, getEditTransaction, updateTransaction, deleteTransaction
CuentasModel	Cuentas	getCounts, getAllcountUser, getCountsUser, setCreateaaccount
PrestamoModel	Prestamos	getPrestamos(id) → COUNT activos
HomeModel	Transacciones / Prestamos / Cuentas	getTotalIngresosMes, getTotalGastosMes, getPrestamosActivos, getCuentas
CategoriasModel	categorias	getCategorias(status)
EstadoModel	estados	getStatus(status)
TipoCuentaModel	Tipo_cuenta	getTypeAccount(status)
TipoMovimientoModel	mm_tipo_movimiento	getTipoMovimiento(status)

4.1 TransacionModel — Detalle

Es el modelo central del sistema. Los métodos de suma (getmontomes, getmontomesegresos) reciben el ID del usuario, una fecha de inicio y una fecha de fin. El orden de parámetros en execute() siempre es: [userId, tipo, fechaInicio, fechaFin].

deleteTransaction(): Implementa Soft Delete — no elimina el registro físicamente. Solo actualiza el campo **estado = 3**. Esto preserva el historial para auditorías.

updateTransaction(): Pendiente de mejora. Actualmente usa set() (INSERT). Debe migrarse a put() cuando el controlador envíe el ID de la transacción a modificar.

5. Controladores

5.1 UserController — Autenticación

Extiende UserModel para acceder a `getUser()` sin instanciar el modelo. Maneja GET (mostrar formulario) y POST (validar credenciales). Las contraseñas se verifican con `password_verify()` comparando contra el hash bcrypt almacenado. El mensaje de error es genérico para no revelar si el usuario existe.

5.2 HomeController — Dashboard

Solo tiene `dashboardAction()`. Carga los 4 catálogos de la BD (tipos de movimiento, categorías, estados, tipos de cuenta) y los pasa como variables PHP a la vista. Los datos transaccionales (balances, gráficas) se cargan luego vía AJAX.

5.3 AjaxController — API Interna JSON

Maneja todas las peticiones AJAX. `App::run()` ya verificó la sesión (HTTP 403 si no hay). El controlador hace una segunda verificación con `$this->userId` como capa adicional. Todas las respuestas son JSON con estructura: `{status, message/datos}`.

Endpoint (action=)	Método	Descripción
getmontomes	POST	Ingresos del mes (rango de fechas)
getmontomesegresos	POST	Egresos del mes (rango de fechas)
getbalancetotal	POST	Balance neto total del usuario
getVerTransacciones	POST	Lista filtrada de transacciones
setTransaction	POST	Registrar nueva transacción
getEditTransaction	POST	Datos de transacción para editar
updateTransaction	POST	Actualizar transacción existente
deleteTransaction	POST	Soft delete (estado = 3)
getCounts	POST	Cuentas activas del usuario
getAllcountUser	POST	Todas las cuentas del usuario
getCountsUser	POST	Cuentas filtradas por tipo y estado
setCreateaaccount	POST	Crear nueva cuenta financiera
getPrestamos	POST	Conteo de préstamos activos

6. Autenticación y Seguridad

6.1 Auth.php — Control de Acceso

Clase estática con 3 métodos. Se llama en el constructor de cada controlador protegido.

Método	Qué hace
Auth::check()	Retorna bool: ¿hay sesión activa?
Auth::allow()	Si NO hay sesión → redirige a la raíz (login)
Auth::role(['admin'])	Si el rol del usuario no está en la lista → redirige

6.2 Medidas de Seguridad Implementadas

Riesgo	Medida implementada
SQL Injection	Prepared Statements con ? en todos los queries
Contraseñas en texto plano	bcrypt via password_hash() / password_verify()
Session Fixation	session.use_only_cookies + session.use_strict_mode
Session Hijacking	Regeneración de ID de sesión cada 30 minutos
Acceso sin autenticar	Auth::allow() en constructores de controladores
AJAX sin sesión	HTTP 403 en App::run() antes de llegar al controlador
Errores expuestos	APP_DEBUG oculta errores en IPs públicas (producción)
URLs manipuladas	Validación de controller/action antes de instanciar clase
Credenciales en repositorio	app/config/app.php en .gitignore + app.example.php

7. Estructura de Base de Datos

Tablas Principales

Tabla	Columnas clave	Descripción
Usuarios	id, nombre, password_hash, activo	Usuarios del sistema
Cuentas	id, usuario_id, nombre, tipo_cuenta, descripcion_estado	Cuentas financieras por usuario
Transacciones	id, usuario_id, cuenta_id, cuenta_destino_id, categoria_id, tipo, monto, descripcion, estado, creado_en	Movimientos financieros
Prestamos	id, usuario_id, estado, ...	Préstamos del usuario

Tablas de Catálogo (Maestros)

Tabla	Columnas	Valores típicos
categorias	id, nombre, estado	Alimentación, Transporte, Salario...
estados	id, tipo_estado, estado	Activo, Inactivo, Pendiente...
Tipo_cuenta	id, nombre, estado	Ahorros, Corriente, Efectivo...
mm_tipo_movimiento	id, tipo_movimiento, estado	1=Ingreso, 2=Egreso, 3=Transferencia

Crear usuario de prueba (contraseña bcrypt)

```
php -r "echo password_hash('demo1234', PASSWORD_BCRYPT);"
-- Insertar el hash resultante en la tabla Usuarios:
```

```
INSERT INTO Usuarios (nombre, password_hash, activo)
VALUES ('demo', '$2y$10$...hash...', 1);
```

8. Patrones de Diseño

Patrón	Archivo	Descripción
Singleton	Connection.php	Una sola conexión PDO para toda la petición
Front Controller	App.php	Punto único de entrada que enruta todas las peticiones
MVC	Estructura general	Separación clara de Models, Views y Controllers
Fluent Interface	Database.php	Métodos encadenables para construir queries
Template Method	head.php + footer.php	Plantillas que envuelven cada vista HTML
Lazy Initialization	AjaxController.php	Modelos instanciados solo cuando se necesitan
Strategy (parcial)	DB_SERVERS array	Múltiples configuraciones de servidor con fallback

9. Instalación

Requisitos

Componente	Versión mínima
PHP	8.1+
Extensión PHP	php_pdo_sqlsrv (habilitar en php.ini)
SQL Server	2016+ o Azure SQL
Servidor web	Apache con mod_rewrite habilitado

Pasos de Instalación

1. Clonar: `git clone https://github.com/tu-usuario/contability-demo.git`
2. Copiar config: `cp app/config/app.example.php app/config/app.php`
3. Editar `app/config/app.php` con las credenciales de la base de datos
4. (Opcional) Configurar variables de entorno: `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS`
5. Apuntar el DocumentRoot del servidor web a la carpeta del proyecto
6. Verificar que `mod_rewrite` esté activo y que `.htaccess` se procese

Variables de entorno recomendadas (.env o servidor):

```
DB_HOST=192.168.1.100\SQLEXPRESS
DB_NAME=ContabilityDB
DB_USER=sa
DB_PASS=tu_contraseña_segura
DB_ENCRYPT=yes
DB_TRUSTED=no
```


10. Mejoras Pendientes (TODO)

Prioridad	Mejora	Archivo
Alta	updateTransaction(): migrar de set() (INSERT) a put() (UPDATE real)	TransacionModel.php
Alta	Validar y castear tipos de \$_POST en AjaxController	AjaxController.php
Alta	Implementar CSRF tokens en formularios AJAX	AjaxController + JS
Media	Unificar nombre de columna: 'activa' vs 'estado' en Cuentas	HomeModel + CuentasModel
Media	Agregar paginación en getVerTransacciones()	TransacionModel + JS
Media	Completar operaciones CRUD de Prestamos	PrestamoModel
Baja	Instalar DataTables y Select2 (rutas definidas en app.php)	app.php / head.php
Baja	Agregar soporte de roles en el login (campo ROL en Usuarios)	UserController